

APOLLO INSTITUTE OF COMPUTER APPLICATION

BACHELOR OF COMPUTER APPLICATION

SEMESTER 03

Object Oriented Programming with Java (BC03001021)

Assignment 2

UNIT 2: Building Blocks of the Language

PART A: 1-MARK QUESTIONS

1. What is type casting?

Answer:

Type casting is the process of **converting a value from one data type to another data type explicitly**.

Example:

```
int a = 10;  
double b = (double) a;
```

2. What is type conversion?

Answer:

Type conversion is the process of **converting a value from one data type to another automatically by Java**.

Example:

```
int a = 10;  
double b = a;
```

3. What is a variable in Java?

Answer:

A variable is a **named memory location used to store a value that can change during program execution**.

Example:

```
int age = 20;
```

4. What is a constant?

Answer:

A constant is a value that **cannot be changed after it is assigned**.

In Java, the `final` keyword is used to declare a constant.

Example:

```
final int MAX = 100;
```

5. What are literals in Java?

Answer:

Literals are **fixed values directly written in a Java program**.

Examples:

```
10          // integer literal
10.5        // floating-point literal
'A'         // character literal
"Hello"     // string literal
true        // boolean literal
```

6. What are identifiers?

Answer:

Identifiers are the **names given to program elements such as variables, methods, classes and objects**.

Example:

```
int marks;
```

Here, `marks` is an identifier.

7. What is an operator?

Answer:

An operator is a **symbol that performs an operation on one or more operands**.

Example:

```
a + b
```

Here, `+` is an operator.

8. What is the *ternary operator*?

Answer:

The ternary operator `?:` is a **conditional operator** that evaluates one of two expressions based on a **condition**.

Example:

```
int max = (a > b) ? a : b;
```

9. What is the *break* statement?

Answer:

The *break* statement is used to **terminate a loop or switch statement immediately**.

Example:

```
break;
```

10. What is the *continue* statement?

Answer:

The *continue* statement is used to **skip the current iteration of a loop and continue with the next iteration**.

Example:

```
continue;
```

11. What is the *return* statement?

Answer:

The *return* statement is used to **exit from a method and optionally return a value to the calling method**.

Example:

```
return sum;
```

12. What is *garbage collection*?

Answer:

Garbage collection is the **automatic process of removing unused objects from memory by the Java Virtual Machine (JVM)**.

13. What is a command-line argument?

Answer:

A command-line argument is a **value passed to a Java program when it is executed from the command line.**

It is received through the `String[] args` parameter of the `main()` method.

14. What is a one-dimensional array?

Answer:

A one-dimensional array is an array that **stores elements of the same data type in a single row or linear structure.**

Example:

```
int marks[] = {10, 20, 30};
```

15. What is a two-dimensional array?

Answer:

A two-dimensional array is an array that **stores elements in rows and columns**, like a table or matrix.

Example:

```
int a[][] = {{1, 2}, {3, 4}};
```

16. What is an array?

Answer:

An array is a **collection of elements of the same data type stored under a single variable name.**

Example:

```
int numbers[] = {10, 20, 30};
```

17. What is the `final` keyword?

Answer:

The `final` keyword is used to **restrict modification**. A final variable cannot be changed, a final method cannot be overridden, and a final class cannot be inherited.

18. Define the *static* keyword.

Answer:

The *static* keyword is used to create a **class-level member that belongs to the class rather than individual objects**.

Example:

```
static int count;
```

PART B: 7-MARK QUESTIONS

1. Explain type conversion and type casting with examples.

Answer:

Type conversion is the process of converting a value from one data type to another. In Java, it can be performed automatically or explicitly.

There are two main types:

1. Implicit Type Conversion

It is also called **widening conversion**. Java automatically converts a smaller data type into a larger compatible data type.

Example:

```
int a = 10;  
double b = a;
```

```
System.out.println(b);
```

Output:

```
10.0
```

The conversion takes place as:

```
byte → short → int → long → float → double
```

It is generally safe because the larger type can represent the smaller type's values.

2. Explicit Type Casting

Type casting is the **explicit conversion of one data type into another using the casting operator**.

The syntax is:

```
dataType variable = (dataType) value;
```

Example:

```
double x = 10.5;
```

```
int y = (int)x;
```

```
System.out.println(y);
```

Output:

10

The decimal part is removed during conversion.

Difference

Type Conversion	Type Casting
Usually performed automatically	Performed explicitly
No casting operator is required	Casting operator is required
Usually widening conversion	Often narrowing conversion
Example: int → double	Example: double → int

Conclusion:

Type conversion allows Java to change data types, while type casting allows the programmer to explicitly specify the required data type.

2. Explain operators in Java in detail.

Answer:

An **operator** is a symbol that performs an operation on one or more operands.

Java provides different types of operators.

1. Arithmetic Operators

Used to perform mathematical operations.

Operator	Meaning
+	Addition
-	Subtraction

Operator	Meaning
*	Multiplication
/	Division
%	Modulus

Example:

```
int a = 10, b = 3;
```

```
System.out.println(a + b);
```

```
System.out.println(a % b);
```

2. Relational Operators

Used to compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

They return either `true` or `false`.

3. Logical Operators

Used to combine conditions.

Operator	Meaning
&&	Logical AND
^	
!	Logical NOT

Example:

```
if (age >= 18 && age <= 60)
```

4. Assignment Operators

Used to assign values to variables.

Examples:

`=, +=, -=, *=, /=, %=`

Example:

```
int a = 10;  
a += 5;
```

5. Unary Operators

Operate on only one operand.

Examples:

`++, --, +, -, !`

Example:

```
int a = 5;  
a++;
```

6. Conditional/Ternary Operator

The ternary operator `?:` is used as a short form of `if-else`.

Example:

```
int max = (a > b) ? a : b;
```

7. Bitwise Operators

Used to perform operations on individual bits.

Examples:

`&, |, ^, ~, <<, >>, >>>`

Conclusion

*Operators make it possible to perform **arithmetic, comparison, logical, assignment and other operations** in Java programs.*

3. Discuss jump statements and their usage.

Answer:

Jump statements are statements used to change the normal flow of execution of a program.

Java provides three main jump statements:

1. *break*
 2. *continue*
 3. *return*
-

1. **break Statement**

The **break** statement is used to **terminate a loop or switch statement immediately**.

Example:

```
for(int i = 1; i <= 5; i++)
{
    if(i == 3)
        break;

    System.out.println(i);
}
```

Output:

1
2

When *i* becomes 3, the loop terminates.

2. **continue Statement**

The **continue** statement **skips the current iteration** and moves to the next iteration of the loop.

Example:

```
for(int i = 1; i <= 5; i++)
{
    if(i == 3)
        continue;

    System.out.println(i);
}
```

Output:

1

2
4
5

The value 3 is skipped.

3. **return** Statement

The **return** statement is used to **exit from a method**. It can also return a value.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Here, **return** sends the result back to the calling method.

Conclusion

Jump statements are useful for **controlling and changing the normal flow of program execution**.

4. Explain decision-making statements in Java with examples.

Answer:

Decision-making statements are used to **execute different blocks of code depending on a condition**.

Java provides the following decision-making statements:

1. **if**
 2. **if-else**
 3. **else-if**
 4. **Nested if**
 5. **switch**
-

1. **if** Statement

It executes a block when the condition is true.

Syntax:

```
if(condition)
{
    // statements
}
```

Example:

```
int age = 20;

if(age >= 18)
{
    System.out.println("Adult");
}
```

2. **if-else Statement**

It executes one block if the condition is true and another block if it is false.

Example:

```
int age = 16;

if(age >= 18)
{
    System.out.println("Adult");
}
else
{
    System.out.println("Minor");
}
```

3. **else-if Ladder**

It is used when there are **multiple conditions**.

Example:

```
int marks = 75;

if(marks >= 80)
    System.out.println("A");
else if(marks >= 60)
    System.out.println("B");
else if(marks >= 40)
    System.out.println("C");
else
    System.out.println("Fail");
```

4. Nested **if**

An **if** statement placed inside another **if** statement is called a nested **if**.

Example:

```
int age = 20;
int citizen = 1;

if(age >= 18)
{
    if(citizen == 1)
        System.out.println("Eligible");
}
```

5. **switch** Statement

The **switch** statement is used to select one block from multiple choices based on the value of an expression.

Example:

```
int day = 2;

switch(day)
{
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    default:
        System.out.println("Invalid day");
}
```

Conclusion

Decision-making statements help a Java program **make choices and execute appropriate statements according to conditions**.

5. Explain looping/iteration statements in Java with examples.

Answer:

Looping statements are used to **execute a block of statements repeatedly** as long as a condition is satisfied.

Java provides three main looping statements:

1. *while*
 2. *do-while*
 3. *for*
-

1. *while* Loop

The *while* loop checks the condition **before executing the loop body**.

Syntax:

```
while(condition)
{
    // statements
}
```

Example:

```
int i = 1;

while(i <= 5)
{
    System.out.println(i);
    i++;
}
```

Output:

```
1
2
3
4
5
```

2. *do-while* Loop

The *do-while* loop executes the body **at least once** because the condition is checked after the loop body.

Syntax:

```
do
{
    // statements
}
while(condition);
```

Example:

```
int i = 1;

do
{
    System.out.println(i);
    i++;
}
while(i <= 5);
```

3. for Loop

The *for* loop is generally used when the number of iterations is known.

Syntax:

```
for(initialization; condition; increment/decrement)
{
    // statements
}
```

Example:

```
for(int i = 1; i <= 5; i++)
{
    System.out.println(i);
}
```

Difference

while	do-while	for
Condition checked first	Condition checked after body	Initialization, condition and update together
May execute zero times	Executes at least once	Useful when iterations are known

Conclusion

Looping statements reduce repetition and make programs **shorter, simpler and more efficient**.

6. Explain the switch statement with an example.

Answer:

The *switch* statement is a **decision-making statement** used to select and execute one block of statements from multiple choices.

Syntax:

```
switch(expression)
{
    case value1:
        // statements
        break;

    case value2:
        // statements
        break;

    default:
        // statements
}
```

Example:

```
int day = 2;

switch(day)
{
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    case 3:
        System.out.println("Wednesday");
        break;

    default:
        System.out.println("Invalid day");
}
```

Output:

Tuesday

Working:

- The value of *day* is 2.
- Java compares it with each *case*.
- *case 2* matches.
- The statement *Tuesday* is printed.
- *break* terminates the *switch*.
- If no case matches, the *default* block is executed.

Conclusion:

The *switch* statement is useful when **one expression needs to be compared with multiple possible values**.

Quick Revision Sheet

1-Mark Keywords

Term	Remember
Type casting	Explicit conversion
Type conversion	Conversion of data type
Variable	Changeable value
Constant	Fixed value
Literal	Fixed value written in code
Identifier	Name of program element
Operator	Performs operation
Ternary	? :
Break	Terminates loop/switch
Continue	Skips current iteration
Return	Exits method/returns value
Garbage collection	Removes unused objects
Command-line argument	Value passed during execution
1-D array	Linear array
2-D array	Rows and columns
Array	Same-type collection
Final	Prevents modification/overriding/inheritance
Static	Belongs to class

